

Shortest Paths in Weighted Graphs

Lecture notes — QTrack demo material

1. The problem

Given a directed graph $G = (V, E)$ with a weight $w(u, v)$ on every edge, the single-source shortest path problem asks for the minimum total weight of a path from a source vertex s to every other vertex. Road networks, packet routing and game AI all reduce to this problem.

A key observation is the optimal substructure property: any subpath of a shortest path is itself a shortest path. This is what makes greedy and dynamic programming approaches possible.

2. Relaxation

Every algorithm in this chapter maintains an estimate $d[v]$ of the distance to each vertex and improves it with a single operation called relaxation. Relaxing an edge (u, v) checks whether going through u gives a shorter path to v , and if so updates $d[v]$ and the predecessor pointer.

```
RELAX( $u, v, w$ ):  
    if  $d[v] > d[u] + w(u, v)$ :  
         $d[v] = d[u] + w(u, v)$   
         $prev[v] = u$ 
```

3. Dijkstra's algorithm

Dijkstra's algorithm works only when all edge weights are non-negative. It repeatedly extracts the vertex with the smallest tentative distance from a min-priority queue and relaxes all of its outgoing edges. Once a vertex is extracted, its distance is final.

With a binary heap the running time is $O((V + E) \log V)$. With a Fibonacci heap it improves to $O(E + V \log V)$, though binary heaps are usually faster in practice.

4. Bellman-Ford

Bellman-Ford handles negative edge weights by relaxing every edge $V - 1$ times, giving $O(VE)$ time. A final pass that still finds an improvement proves the graph contains a negative-weight cycle reachable from the source, in which case no shortest path exists.

5. Choosing an algorithm

Use BFS for unweighted graphs, Dijkstra for non-negative weights, Bellman-Ford when negative edges are possible, and Floyd-Warshall when you need all-pairs distances on a small dense graph.